



JavaScript Practical Workbook

Topics 05 – 14 · Short Examples & Exercises

Variables	Data Types	Operators	Functions	Loops	Conditions	Arrays	Objects	Events	DOM
-----------	------------	-----------	-----------	-------	------------	--------	---------	--------	-----

TOPIC 05 Variables

Variables store data values. JavaScript has three declaration keywords: `var` (function-scoped, legacy), `let` (block-scoped, reassignable), and `const` (block-scoped, constant binding).

■ Example 1 – `var`, `let`, `const` basics

```
var   city   = 'Mumbai';
let   age    = 21;
const PI     = 3.14159;

console.log(city); // Mumbai
console.log(age);  // 21
console.log(PI);   // 3.14159
```

■ Example 2 – Reassignment & block scope

```
let score = 10;
score = 20;           // OK - let is reassignable

{
  let inner = 'block';
  console.log(inner); // block
}
// console.log(inner); // ReferenceError - out of scope
console.log(score);   // 20
// Output: block 20
```

■ Example 3 – `const` with objects

```
const person = { name: 'Ayesha' };
person.name = 'Riya'; // allowed - mutating the object
console.log(person.name); // Riya
// person = {};        // TypeError - can't reassign const
// Output: Riya
```

TOPIC 06 Data Types

JavaScript has 7 primitive types (String, Number, BigInt, Boolean, undefined, null, Symbol) and one complex type: Object (which includes arrays and functions).

■ Example 1 – Primitive types

```
let str = 'Hello';           // String
let num = 42;                 // Number
let big = 9007199254740993n; // BigInt
let bool = true;              // Boolean
let undef;                    // undefined
let empty = null;             // null

console.log(typeof str, typeof num); // string number
console.log(typeof big, typeof bool); // bigint boolean
console.log(typeof undef, typeof empty); // undefined object
// Output: string number bigint boolean undefined object
```

■ Example 2 – Type coercion

```
console.log('5' + 3); // '53' (string concat)
console.log('5' - 3); // 2 (numeric subtract)
console.log(true + 1); // 2
console.log(null + 1); // 1
// Output: 53 2 2 1
```

■ Example 3 – typeof & strict equality

```
console.log(0 == false); // true (loose)
console.log(0 === false); // false (strict - diff types)
console.log(typeof null); // 'object' (JS quirk!)
// Output: true false object
```

TOPIC 07 Operators

JavaScript operators perform arithmetic, comparison, logical, assignment, and other operations on values.

■ Example 1 – Arithmetic operators

```
let a = 10, b = 3;
console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1 (remainder)
console.log(a ** b); // 1000 (exponent)
// Output: 13 / 7 / 30 / 3.333... / 1 / 1000
```

■ Example 2 – Comparison & logical

```
console.log(5 > 3); // true
console.log(5 === '5'); // false
console.log(true && false); // false
console.log(true || false); // true
console.log(!true); // false
// Output: true false false true false
```

■ Example 3 – Nullish coalescing & optional chaining

```
let user = null;
console.log(user ?? 'Guest'); // Guest

let obj = { profile: { name: 'Ali' } };
console.log(obj?.profile?.name); // Ali
console.log(obj?.address?.city); // undefined
// Output: Guest Ali undefined
```

TOPIC 08 Functions

Functions are reusable blocks of code. JavaScript supports function declarations, expressions, arrow functions, default parameters, and rest/spread.

■ Example 1 – Declaration vs expression vs arrow

```
// Declaration
function greet(name) {
  return 'Hello, ' + name + '!';
}

// Expression
const square = function(x) { return x * x; };

// Arrow function
const cube = x => x ** 3;

console.log(greet('World')); // Hello, World!
console.log(square(4));      // 16
console.log(cube(3));        // 27
// Output: Hello, World! 16 27
```

■ Example 2 – Default & rest parameters

```
function add(a, b = 10) {
  return a + b;
}

console.log(add(5));      // 15
console.log(add(5, 3));   // 8

function sum(...nums) {
  return nums.reduce((t, n) => t + n, 0);
}

console.log(sum(1, 2, 3, 4)); // 10
// Output: 15 8 10
```

■ Example 3 – Higher-order & closure

```
function multiplier(factor) {
  return num => num * factor; // closure
}

const double = multiplier(2);
const triple = multiplier(3);
console.log(double(5)); // 10
console.log(triple(5)); // 15
// Output: 10 15
```

TOPIC 09 Loops

Loops repeatedly execute a block of code. JavaScript offers for, while, do...while, for...of, and for...in loops.

■ Example 1 – for & while

```
// for loop
for (let i = 1; i <= 3; i++) {
  console.log('for:', i);
}

// while loop
let n = 3;
while (n > 0) {
  console.log('while:', n--);
}
// Output: for:1 for:2 for:3 while:3 while:2 while:1
```

■ Example 2 – for...of & for...in

```
const fruits = ['apple', 'mango', 'banana'];

// for...of – iterate values
for (const fruit of fruits) {
  console.log(fruit);
}

// for...in – iterate keys/indices
const car = { brand: 'Honda', year: 2022 };
for (const key in car) {
  console.log(key, ': ', car[key]);
}
// Output: apple mango banana brand:Honda year:2022
```

■ Example 3 – break & continue

```
for (let i = 0; i < 6; i++) {
  if (i === 2) continue; // skip 2
  if (i === 5) break;    // stop at 5
  console.log(i);        // 0 1 3 4
}
// Output: 0 1 3 4
```

TOPIC 10 Conditions

Conditional statements control the flow of code based on boolean expressions using if/else, switch, and the ternary operator.

■ Example 1 – if / else if / else

```
let marks = 72;

if (marks >= 90) {
  console.log('A Grade');
} else if (marks >= 60) {
  console.log('B Grade');
} else {
  console.log('C Grade');
}

// Output: B Grade
```

■ Example 2 – switch statement

```
let day = 'Mon';

switch (day) {
  case 'Mon': console.log('Monday'); break;
  case 'Tue': console.log('Tuesday'); break;
  default:   console.log('Other day');
}

// Output: Monday
```

■ Example 3 – Ternary & short-circuit

```
let age = 20;
let status = age >= 18 ? 'Adult' : 'Minor';
console.log(status); // Adult

// Short-circuit evaluation
let name = '' || 'Anonymous';
console.log(name); // Anonymous

let isAdmin = true;
isAdmin && console.log('Welcome, Admin!');

// Output: Adult Anonymous Welcome, Admin!
```

TOPIC 11 Arrays

Arrays store ordered collections. JavaScript arrays are dynamic, zero-indexed, and come with powerful built-in methods.

■ Example 1 – Create, access, modify

```
const nums = [10, 20, 30, 40];

console.log(nums[0]);    // 10
console.log(nums.length); // 4

nums.push(50);           // add at end
nums.unshift(0);         // add at start
nums.pop();              // remove last
console.log(nums);       // [0,10,20,30,40]
// Output: [0, 10, 20, 30, 40]
```

■ Example 2 – map, filter, reduce

```
const prices = [100, 250, 80, 430];

const doubled = prices.map(p => p * 2);
const cheap   = prices.filter(p => p < 200);
const total   = prices.reduce((s, p) => s + p, 0);

console.log(doubled); // [200,500,160,860]
console.log(cheap);  // [100,80]
console.log(total);   // 860
// Output: [200,500,160,860] [100,80] 860
```

■ Example 3 – Spread, destructuring & find

```
const a = [1, 2, 3];
const b = [4, 5, 6];
const combined = [...a, ...b];
console.log(combined); // [1,2,3,4,5,6]

const [first, second, ...rest] = combined;
console.log(first, second, rest);
// 1 2 [3,4,5,6]
// Output: 1 2 [3,4,5,6]
```

TOPIC 12 Objects

Objects are key-value collections. They model real-world entities and support methods, destructuring, and prototype-based inheritance.

■ Example 1 – Create & access

```
const student = {
  name: 'Priya',
  age: 20,
  greet() {
    return `Hi, I'm ${this.name}`;
  }
};

console.log(student.name);    // Priya
console.log(student['age']);  // 20
console.log(student.greet()); // Hi, I'm Priya
// Output: Priya 20 Hi, I'm Priya
```

■ Example 2 – Destructuring & spread

```
const { name, age, city = 'Mumbai' } = student;
console.log(name, age, city); // Priya 20 Mumbai

const updated = { ...student, age: 21 };
console.log(updated.age);     // 21
// Output: Priya 20 Mumbai 21
```

■ Example 3 – Object methods

```
const car = { brand: 'Toyota', year: 2022 };

console.log(Object.keys(car));
// ['brand', 'year']

console.log(Object.values(car));
// ['Toyota', 2022]

console.log(Object.entries(car));
// [['brand', 'Toyota'], ['year', 2022]]
// Output: ['brand', 'year'] ['Toyota', 2022]
```

TOPIC 13 Events

Events allow JavaScript to respond to user interactions. Listeners are attached via `addEventListener` and can handle clicks, keyboard input, form submissions, and more.

■ Example 1 – Click event

```
<button id='btn'>Click Me</button>

const btn = document.getElementById('btn');

btn.addEventListener('click', function() {
  alert('Button clicked!');
});
// Output: Alert: Button clicked!
```

■ Example 2 – Mouse & keyboard events

```
document.addEventListener('keydown', (e) => {
  console.log('Key pressed:', e.key);
});

const box = document.querySelector('.box');
box.addEventListener('mouseover', () => {
  box.style.background = 'yellow';
});
box.addEventListener('mouseout', () => {
  box.style.background = '';
});
// Output: Key pressed: Enter
```

■ Example 3 – Event delegation

```
<ul id='list'>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>

document.getElementById('list')
  .addEventListener('click', (e) => {
    if (e.target.tagName === 'LI') {
      console.log('Clicked:', e.target.textContent);
    }
  });
// Output: Clicked: Item 2
```

TOPIC 14 DOM Manipulation

The Document Object Model (DOM) represents the HTML document as a tree. JavaScript can select, create, modify, and delete elements dynamically.

■ Example 1 – Select & modify elements

```
// Select elements
const h1 = document.querySelector('h1');
const paras= document.querySelectorAll('p');

// Modify content & style
h1.textContent = 'Hello DOM!';
h1.style.color = 'royalblue';

paras.forEach(p => p.classList.add('highlight'));
// Output: Page heading changes to 'Hello DOM!'
```

■ Example 2 – Create & append elements

```
const ul= document.createElement('ul');

['JS', 'HTML', 'CSS'].forEach(lang => {
  const li = document.createElement('li');
  li.textContent = lang;
  ul.appendChild(li);
});

document.body.appendChild(ul);
// Adds a list of languages to the page
// Output: List with JS, HTML, CSS added
```

■ Example 3 – Remove & replace elements

```
const old = document.querySelector('#old');

// Remove
old.remove();

// Replace
const newEl = document.createElement('p');
newEl.textContent = 'New paragraph!';

const parent = document.querySelector('#container');
const target = document.querySelector('#target');
parent.replaceChild(newEl, target);
// Output: Old element removed; new paragraph inserted
```